KAPASA MAKASA UNIVERSITY
DATA SCIENCE YEAR TWO
ICT 220/ICT 250 DATA STRUCTURE AND ALGORITHMS CLASS ASSIGNMENT

Each group must develop an optimized C++ solution to its assigned computational problem, applying the principles of Algorithm Analysis and Algorithm Design Techniques.

Instructions to Students

1. Define and explain the concepts: Before presenting the program, each group must define the relevant algorithm analysis and design concepts and explain how they apply to the assigned problem.
2. Design the algorithm: Write the algorithm using pseudocode or a flowchart and identify the design technique used, such as Greedy, Divide and Conquer, Dynamic Programming, or Backtracking.
3. Implement in C++: Develop and demonstrate a complete, working program that solves the assigned problem.
4. Analyze algorithm efficiency: Determine the time and space complexity, best-, worst-, and average-case performance, and calculate the running time.
5. Evaluate and optimize: Explain the efficiency of the solution, compare alternative approaches where applicable, and justify any optimization made.
6. Present and defend: Demonstrate the program, explain the results, and answer technical questions from the lecturer and classmates.
7. Total marks [10].

Allocation of Algorithm Design Techniques

GROUPS	**DESIGN TECHNIQUE**
1-3	GREEDY Algorithms
4-6	Divide and Conguer
7-8	Dynamic Programming
9-10	Backtracking

Each group must relate its assigned programming problem to the designated algorithm design technique and demonstrate how the technique contributes to an efficient solution.

Presentation date Friday 17th September,2026/duration: 15–20 minutes per group.

Group 1: Design and implement a C++ program that manages the academic performance of students across multiple subjects.
The program must accept student details, subject marks, and credit hours. It should calculate weighted averages, assign grades, rank students, and identify students who require academic support.
Requirements:

- Use arrays, strings, loops, conditional statements, and functions.
- Implement a searching algorithm to locate a student by identification number.
- Implement a sorting algorithm to rank students according to their weighted averages.
- Analyze the best, worst, and average cases of the searching and sorting algorithms.
- Calculate the time and space complexity of the complete solution.

Presentation: Demonstrate the program using at least 10 student records and explain why the selected sorting algorithm is efficient.

Group 2: Develop a C++ library management system that stores and processes information about books, authors, ISBN numbers, and borrowing status.
The program must allow librarians to add books, search for books, borrow and return books, and display overdue books based on borrowing periods.
Requirements:

- Use arrays or appropriate data structures to store book records.
- Implement linear search and binary search for locating books by ISBN.
- Sort book records before applying binary search.
- Compare the time complexity of linear and binary search.
- Analyze the space complexity of storing and processing book records.

Presentation: Demonstrate both searching algorithms and explain the conditions under which binary search is more efficient.

Group 3: Design and implement a C++ banking system that manages customer accounts and financial transactions.
The program should support account creation, deposits, withdrawals, balance inquiries, and transaction history. It must reject invalid withdrawals and maintain accurate account balances.
Requirements:

- Use structures or classes to represent customer accounts.
- Implement functions for each banking operation.
- Search for accounts using an identification number.
- Analyze the running time of account searching and transaction processing.
- Evaluate the space complexity of storing transaction histories.

Presentation: Demonstrate successful and unsuccessful transactions, explain the algorithms, and identify possible efficiency improvements.

Group 4: Develop a C++ program for managing hospital patient records, including patient identification numbers, names, ages, medical conditions, and appointment information.
The program must support patient registration, record searching, updating patient information, and generating a list of patients requiring urgent attention based on defined priority levels.
Requirements:

- Use structures or classes to represent patient records.
- Implement searching and sorting algorithms.
- Develop an algorithm for prioritizing patients.
- Analyze the best, worst, and average cases of searching and sorting.
- Determine the time and auxiliary space complexity of the program.

Presentation: Demonstrate patient registration, searching, and priority ordering using sample records.

Group 5: Design and implement a C++ inventory management system for a retail business. The program must manage product codes, names, prices, quantities, and reorder levels. It should process sales, update stock, calculate total inventory value, and identify products requiring restocking.

Requirements:

- Use arrays, structures, or classes for product records.
- Implement product searching using linear search and binary search.
- Develop an algorithm for sorting products by price or quantity.
- Analyze the running time of stock updates and product searching.
- Evaluate the space complexity of the inventory data structure.

Presentation: Demonstrate stock updates, product searches, and restocking alerts. Justify the choice of algorithms.

Group 6: Develop a C++ program that calculates electricity bills for customers based on units consumed and a defined tariff structure.

The program should store customer details, calculate charges using progressive tariff rates, apply fixed charges where applicable, and generate monthly billing reports.

Requirements:

- Implement the billing algorithm using functions and conditional statements.
- Process multiple customers using arrays or structures.
- Calculate the total revenue from all customers.
- Analyze the running time of billing calculations for n customers.
- Determine the space complexity of storing customer billing records.

Presentation: Demonstrate billing calculations for different consumption levels and explain the efficiency of the algorithm.

Group 7: Design and implement a C++ attendance management system that records student attendance for multiple classes.

The program must calculate attendance percentages, identify students below a specified attendance threshold, and generate a report of students at academic risk.

Requirements:

- Use two-dimensional arrays or appropriate data structures.
- Implement functions for recording attendance and calculating percentages.
- Search for students using identification numbers.
- Analyze the time complexity of processing attendance for n students across mmm classes.
- Evaluate the space complexity of the attendance matrix.

Presentation: Demonstrate attendance recording, report generation, and the effect of increasing the number of students and classes on running time.

Group 8: Develop a C++ examination system that presents multiple-choice questions, accepts student answers, calculates scores, and generates performance reports.

The program should support multiple students and allow the lecturer to store questions, correct answers, and marks.
Requirements:

- Use arrays, strings, loops, and functions.
- Implement an algorithm for comparing submitted answers with correct answers.
- Calculate individual scores and class averages.
- Sort students according to their examination scores.
- Analyze the time and space complexity of answer checking and student ranking.

Presentation: Demonstrate the examination process, automated marking, and performance ranking.

Group 9: Design and implement a C++ payroll system that calculates employees' gross salaries, allowances, deductions, and net salaries.
The program must store employee details, process payroll for multiple employees, and generate individual payslips and a summary report.
Requirements:

- Use structures or classes to store employee records.
- Implement functions for salary calculations and deductions.
- Search for employees by identification number.
- Sort employees by net salary or department.
- Analyze the running time and space complexity of payroll processing.

Presentation: Demonstrate payroll calculations, employee searching, and sorting. Explain the efficiency of the selected algorithms.

Group 10: Develop a C++ transport ticket booking system that manages passengers, bus routes, seat availability, and ticket payments.
The program must allow passengers to book seats, cancel reservations, search for available seats, and generate booking reports.
Requirements:

- Use arrays or suitable data structures to represent seats and passenger records.
- Implement seat searching and allocation algorithms.
- Develop an algorithm for booking cancellation and seat availability updates.
- Analyze the best, worst, and average cases of seat searching.
- Calculate the time and space complexity of the booking system.

Presentation: Demonstrate seat booking, cancellation, searching, and the efficiency of the algorithms used.